SHERLOCK

Security Review For Celestia

Collaborative Audit Prepared For:	**Celestia**
Lead Security Expert(s):	**g** **JakubHeba**
Date Audited:	**January 12 - January 16, 2026**

Introduction

The proposal this security review focuses on, introduces a ZK-based Hyperlane Interchain Security Module (x/zkism) that enables trust-minimized cross-chain message verification using Groth16 zero-knowledge proofs. The module allows external networks to securely transfer assets through Celestia without relying on trusted operators or multisigs.

Scope

Repository: celestiaorg/celestia-app

Audited Commit: 7f23efe206642d6101f6acc30f21ef1ef135a2d8

Final Commit: d3e13fa1844405c2e3b358fe9c5a9a25745b3b19

Files:

- proto/celestia/zkism/v1/events.proto
- proto/celestia/zkism/v1/genesis.proto
- proto/celestia/zkism/v1/query.proto
- proto/celestia/zkism/v1/tx.proto
- proto/celestia/zkism/v1/types.proto
- x/zkism/client/cli/cli.go
- x/zkism/client/cli/query.go
- x/zkism/client/cli/tx.go
- x/zkism/doc.go
- x/zkism/internal/groth16/groth16.go
- x/zkism/keeper/events.go
- x/zkism/keeper/genesis.go
- x/zkism/keeper/grpc_query.go
- x/zkism/keeper/keeper.go
- x/zkism/keeper/msg_server.go
- x/zkism/module.go
- x/zkism/README.md
- x/zkism/types/codec.go
- x/zkism/types/errors.go
- x/zkism/types/expected_interfaces.go
- x/zkism/types/genesis.go

- x/zkism/types/ism.go
- x/zkism/types/keys.go
- x/zkism/types/msgs.go
- x/zkism/types/public_values.go
- x/zkism/types/verifier.go

Final Commit Hash

d3e13fa1844405c2e3b358fe9c5a9a25745b3b19

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	2	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Unbounded memory allocation in `StateTransitionValues.Unmarshal` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/issues/32

Summary

Attacker-controlled length prefix in public values deserialization triggers unbounded memory allocation before validation, causing validator crashes and chain halt.

Vulnerability Detail

The `StateTransitionValues.Unmarshal` function deserializes the `public_values` field from `MsgUpdateInterchainSecurityModule` transactions using a bincode-compatible format with length-prefixed byte arrays. The function reads an 8-byte little-endian unsigned integer from the input buffer to determine the length of the `State` field, then immediately allocates a byte slice of that size before attempting to read data into it.

The issue is that the allocation occurs before any validation against the actual remaining bytes in the buffer. An attacker can craft a transaction with a minimal `public_values` payload where the first 8 bytes encode an extremely large length value while the actual payload contains only a few bytes. When validators process this transaction during block execution, the Go runtime attempts to allocate the requested memory.

For allocation requests that exceed available system memory, Go's runtime calls an internal throw function rather than panic. Unlike panic, throw cannot be caught by defer mechanisms. Cosmos SDK's `BaseApp.runTx` includes panic recovery middleware, but this protection is ineffective against runtime throws. The process terminates immediately without executing any deferred cleanup or error handling. This results in simultaneous termination of all validator processes, halting consensus.

The same vulnerability pattern exists in the `NewState` field deserialization within the same function, as well as in `StateMembershipValues.Unmarshal` where the `MessageIds` array length is read and allocated without bounds checking.

Impact

A single malicious transaction can halt the blockchain. All validators crash simultaneously when processing the block containing the malicious transaction.

Code Snippet

```go
// Unmarshal decodes a bincode-serialized PublicValues struct.
// This function expects the input byte slice to be encoded using Rust bincode's
// default configuration: (little-endian, fixed-width integers, length-prefixed
↪ slices).
func (v *StateTransitionValues) Unmarshal(data []byte) error {
    buf := bytes.NewReader(data)

    // read length of State
    var l uint64
    if err := binary.Read(buf, binary.LittleEndian, &l); err != nil {
        return err
    }

    // read State
    v.State = make([]byte, l)
    if _, err := io.ReadFull(buf, v.State); err != nil {
        return err
    }

    // read length of NewState
    var newL uint64
    if err := binary.Read(buf, binary.LittleEndian, &newL); err != nil {
        return err
    }

    // read NewState
    v.NewState = make([]byte, newL)
    if _, err := io.ReadFull(buf, v.NewState); err != nil {
        return err
    }

    return nil
}
```

Tool Used

Manual Review

Recommendation

Validate the decoded length against the remaining buffer size before performing any allocation. Reject lengths that exceed available data. Additionally, enforce a protocol-level maximum size for state values to prevent resource exhaustion even with correctly-formed inputs.

Issue M-1: Lacking validation on the State's length when creating an ISM [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/issues/26

Summary

An ISM's state must be at least 32 bytes, because its first 32 bytes is the state root. However, there are is no validation that this is the case when creating the ISM.

Vulnerability Detail

The CreateInterchainSecurityModule handler does not validate the length of the ISM's state.

Impact

- can't update ISM because of validation that expects it to be 32 bytes.
- can't submit messages because it would panic if less than 32 bytes.

Code Snippet

https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/blob/main/celestia-app/x/zkism/keeper/msg_server.go#L26-L53

Tool Used

Manual Review

Recommendation

Consider implementing the validation that the ISM's state is at least 32 bytes when creating the ISM.

Issue M-2: Anyone can exhaust node resources with queries due to lacking pagination limit restrictions [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/issues/27

Summary

The query handler for an ISM's messages does not implement any restrictions on the pagination limit. The pagination limit is the number of messages returned by the node's query server. Since anyone can call this query, this makes the node vulnerable to DOS.

Vulnerability Detail

Lacking restrictions on the pagination limit allows users to set it to a large value.

```
func (q queryServer) Messages(ctx context.Context, req *types.QueryMessagesRequest)
↪  (*types.QueryMessagesResponse, error) {
        // ... snip ...

    msgs, pageRes, err := query.CollectionPaginate(
        ctx,
        q.k.messages,
        req.Pagination,
        transformFunc,
        query.WithCollectionPaginationPairPrefix[uint64,
        ↪  []byte](ismId.GetInternalId()),
    )
```

This enables anyone to send expensive queries that drains the node's resources once an ISM has many messages.

Impact

Anyone can DOS the node through resource exhaustion.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider implementing restriction on the pagination limit.

Discussion

damiannolan

I believe this is true for all standard cosmos-sdk queries, since no maximum seems to be enforced

gjaldon

@damiannolan yes, this is true for all cosmos-sdk queries and it will be a problem for large collections.

damiannolan

I would rather recommend addressing this at the cosmos sdk level rather than in the module query. Since celestia maintains a dedicated fork, I think this should be no problem. I will bring this up with the celestia core team. Thank you

Issue L-1: Lacking validation in MerkleTreeAddress when creating ISM [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/issues/28

Summary

When Submitting Messages for an ISM, the Merkle Tree address is expected to be a 32-byte slice. However, this expectation is not enforced when creating an ISM.

Vulnerability Detail

There is no validation that the Merkle Tree address has a length of 32 bytes when creating the Interchain Security Module.

Impact

Submitting messages for the ISM will always fail because of this check.

Code Snippet

https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/blob/main/celestia-app/x/zkism/keeper/msg_server.go#L26-L40

Tool Used

Manual Review

Recommendation

Consider adding validation that the length of Merkle Tree address is 32 bytes when creating the ISM.

Issue L-2: Anyone can resend the same proof and set of messages via `SubmitMessages()` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/issues/29

Summary

Once a message is verified, it is removed from the store. However, there is nothing preventing the submission of the same set of messages and proof to re-add the same message that has already been verified.

This allows anyone to add the same messages that were already verified back to the messages store.

Vulnerability Detail

There is no replay protection in the message submission handler in ZKISM.

Impact

Hyperlane's Mailbox has replay protection when processing messages. However, the issue can make `VerifyDryRun()` query succeed when it should fail.

Code Snippet

https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/blob/main/celestia-app/x/zkism/keeper/msg_server.go#L100-L145

Tool Used

Manual Review

Recommendation

Consider adding replay protection in the message submission handler.

Issue L-3: Wrong arguments used in `SubmitMessagesCmd` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/issues/30

Summary

The submit-messages CLI command reads arguments from wrong indices, causing runtime panic on execution.

Vulnerability Detail

The `NewSubmitMessagesCmd` function in the CLI package defines a command that accepts exactly three positional arguments: the ISM identifier, the hex-encoded proof bytes, and the hex-encoded public values. The command specification correctly documents this interface and enforces the argument count through Cobra's `ExactArgs` validator.

However, the implementation incorrectly accesses these arguments using wrong array indices. The proof bytes are read from index 2 instead of index 1, and the public values are read from index 3 instead of index 2. Since the command only accepts three arguments, valid indices are 0, 1, and 2. Accessing index 3 exceeds the array bounds.

When an operator attempts to use this command with the correct number of arguments, the Go runtime raises an index out of bounds error. This occurs before any transaction construction or network communication, making the command non-functional for its intended purpose.

Impact

Operators cannot use the command-line interface to submit message proofs to the zkism module. This forces reliance on alternative submission methods such as direct gRPC calls or programmatic transaction construction.

Code Snippet

```go
// NewSubmitMessagesCmd creates and returns the submit messages cmd.
func NewSubmitMessagesCmd() *cobra.Command {
    cmd := &cobra.Command{
        Use:   "submit-messages [ism-id] [proof-hex] [public-values-hex]",
        Short: "Submit a batched Hyperlane message proof for an ISM",
        Long: strings.TrimSpace(`Submit a Hyperlane zk membership proof to
        ↪  authorize message IDs for processing.
```

```
The command submits a Groth16 state membership proof and public values against an
↪  existing ISM.

Arguments:
  [ism-id]             Hex-encoded 32-byte ISM identifier (0x-prefixed is accepted).
  [proof-hex]          Hex-encoded SP1 Groth16 proof bytes (260 bytes: 4-byte
  ↪  prefix + 256-byte proof).
  [public-values-hex]  Hex-encoded bincode-serialized StateMembershipValues
  ↪  (state_root || merkle_tree_address || message_ids).`),
        Example: fmt.Sprintf("%s tx %s submit-messages
         ↪  0x726f757465725f69736d0000000000000000000000000002a0000000000000000
         ↪  0xdead...beef 0xdead...beef", version.AppName, types.ModuleName),
        Args:    cobra.ExactArgs(3),
        RunE: func(cmd *cobra.Command, args []string) error {
            clientCtx, err := client.GetClientTxContext(cmd)
            if err != nil {
                return err
            }

            ismID, err := util.DecodeHexAddress(args[0])
            if err != nil {
                return fmt.Errorf("invalid ism identifier: %w", err)
            }

            proof, err := decodeHexString(args[2])
            if err != nil {
                return fmt.Errorf("invalid proof: %w", err)
            }

            publicValues, err := decodeHexString(args[3])
            if err != nil {
                return fmt.Errorf("invalid public values: %w", err)
            }

            msg := types.MsgSubmitMessages{
                Id:           ismID,
                Proof:        proof,
                PublicValues: publicValues,
                Signer:       clientCtx.GetFromAddress().String(),
            }

            return tx.GenerateOrBroadcastTxCLI(clientCtx, cmd.Flags(), &msg)
        },
    }

    flags.AddTxFlagsToCmd(cmd)
    return cmd
}
```

Tool Used

Manual Review

Recommendation

Correct the argument indices so that proof is read from index 1 and public values from index 2.

Issue L-4: Missing ISM identifier in `SubmitMessages` event emission [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-celestia-zk-ism-verifier-jan-12th/issues/31

Summary

The `EventSubmitMessages` event omits the ISM identifier field, preventing off-chain systems from associating authorized messages with their source ISM.

Vulnerability Detail

The zkism module emits typed events after successful transaction execution to enable off-chain observability. The `EventSubmitMessages` protobuf definition includes three fields - the ISM identifier, the state root used for verification, and the list of authorized message identifiers. This event is emitted after a successful `SubmitMessages` transaction to signal that specific Hyperlane messages have been authorized for processing.

The `EmitSubmitMessagesEvent` function constructs and emits this event but only populates the state root and message list fields. The ISM identifier field is never set, resulting in an empty or zero-value identifier in all emitted events. This contrasts with the `EmitCreateISMEvent` and `EmitUpdateISMEvent` functions which correctly populate the identifier field.

Impact

Event-driven architectures and off-chain infrastructure cannot function correctly in deployments with multiple ISMs.

Code Snippet

```
message EventSubmitMessages {
  option (gogoproto.goproto_getters) = false;

  // unique hyperlane identifier
  string id = 1 [
    (gogoproto.customtype) =
    ↪  "github.com/bcp-innovations/hyperlane-cosmos/util.HexAddress",
    (gogoproto.nullable)   = false
  ];
  // state commitment root used to verify messages (hex-encoded)
  string state_root = 2;
  // list of authorized messages ids
```

```
  repeated string messages = 3;
}
```

Tool Used

Manual Review

Recommendation

Modify the `EmitSubmitMessagesEvent` function to accept the ISM identifier as an additional parameter and populate the `Id` field in the constructed event. Update the call site in the `SubmitMessages` handler to pass the ISM identifier from the retrieved ISM object.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.